

Hesham Rashid

ISM 6208 – Data Warehousing

University of South Florida

March 15th, 2026

Professor Don Berndt

Part B (Optional): Dimensional Model for Data of Your Choice:

The main analytical question that drives this design is: How do grocery store data analysts and engineers make decisions for their store?

Part B Report:

Quick Introduction:

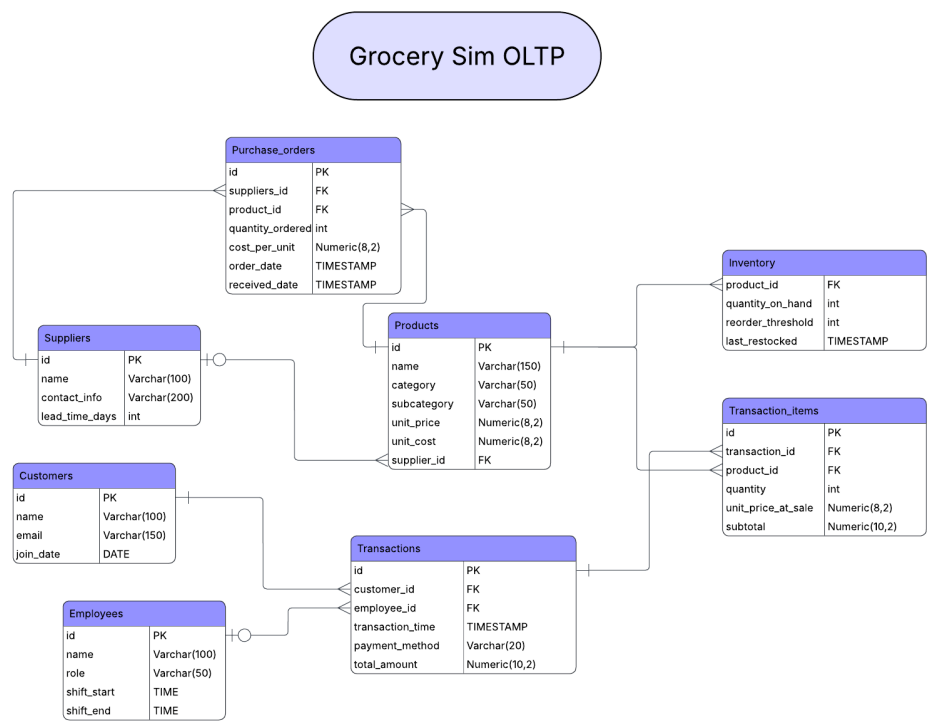
This is a personal project I have done on the side. It is a grocery store simulation. There are only 44 things to buy in the store, its like a mom and pop type of corner store, there are no chains, support local business! There are two databases there is an OLTP database that handles all the transactional data and then there is an OLAP database that I can do analytical queries on. Note that both of these databases are on PostgreSQL. The simulation itself is all python scripts, there is a seeder script that outlines things like stuff to buy and employees at the beginning of the simulation. There is an actual simulation script that does the simulation based off probabilities.

There is an ETL script that does the ETL process. It opens connections to both databases at the same time then runs a SQL query on the OLTP database that joins the relevant tables (transactional_items, transactions, products, customers, employees) into a big flat list of every item sold that day, then it will populate the dimension tables in the OLAP database (dim_time, dim_product, dim_customer, dim_employee, dim_payment_method). Then it will build loop up maps, to make sure the OLTP source IDs and the OLAP dim IDs match. So product_id from the OLTP database matches the dim_product_id from the OLAP. Then we insert the data into the fact_sales table. The final step is the script then resets the OLTP database, so that it clears out the data. This script runs every simulated day, so the OLTP database is always fresh and can quickly take in transactions, and the OLAP has updated data.

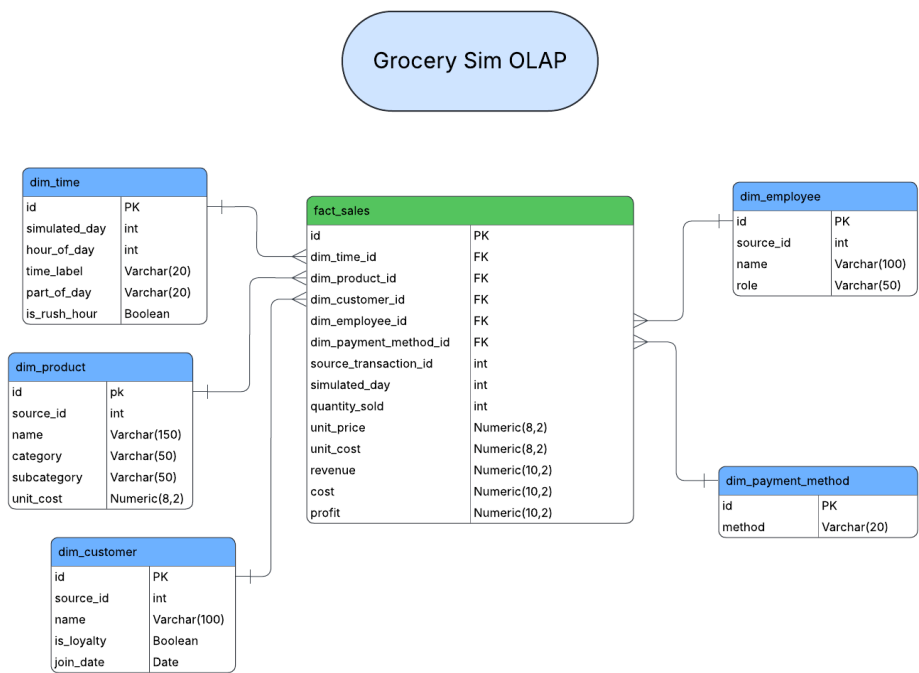
This is a link to my GitHub repository for this project, it has the scripts, the ERDs, the schemas, and the report queries (I will show a few below with data):

<https://github.com/hrashid13/grocery-sim>

This is the schema of the OLTP database:



This is the schema of the OLAP database:



Analytical Queries:

I have a script file that has 10 queries, some more interesting than others. I will show a few here.

Query 1: Top 10 best selling products:

```
SELECT
    dp.name                                AS product,
    dp.category,
    SUM(fs.quantity_sold)                 AS total_units_sold,
    ROUND(SUM(fs.revenue)::numeric, 2)    AS total_revenue,
    ROUND(SUM(fs.profit)::numeric, 2)     AS total_profit,
    ROUND((SUM(fs.profit) / NULLIF(SUM(fs.revenue), 0) * 100)::numeric, 1) AS profit_margin_pct
FROM fact_sales fs
JOIN dim_product dp ON fs.dim_product_id = dp.id
GROUP BY dp.name, dp.category
ORDER BY total_revenue DESC
LIMIT 10;
```

dim_product 1 X

SELECT dp.name AS product, dp.category, S... Enter a SQL expression to filter results (use Ctrl+Space)

	AZ product	AZ category	123 total_units_sold	123 total_revenue	123 total_profit	123 profit_margin_pct
1	Coffee (12oz ground)	Beverages	89,517	804,757.83	375,076.23	46.6
2	Coca-Cola (12pk cans)	Beverages	89,340	713,826.6	311,796.6	43.7
3	Olive Oil (16oz)	Pantry	88,393	706,260.07	335,009.47	47.4
4	Chicken Nuggets (24oz)	Frozen	89,388	669,516.12	329,841.72	49.3
5	Frozen Pizza (pepperoni)	Frozen	89,132	623,032.68	311,070.68	49.9
6	Bacon (12oz)	Meat	89,102	622,822.98	284,235.38	45.6
7	Chicken Breast (lb)	Meat	89,194	534,272.06	248,851.26	46.6
8	Greek Yogurt (32oz)	Dairy	88,339	529,150.61	246,465.81	46.6
9	Ground Beef 80/20 (lb)	Meat	90,620	497,503.8	225,643.8	45.4
10	Butter (1lb)	Dairy	89,333	472,571.57	195,639.27	41.4

Very straightforward query. This shows that coffee is our best selling product by total revenue.

What is interesting to note is that even though coffee sells the most, and does bring the most total profit, it is not the best profit margin, that would go to frozen pizza and chicken nuggets which are both frozen items that are interesting to note.

Query 2: payment method break down

```
SELECT
    dpm.method AS payment_method,
    COUNT(DISTINCT fs.source_transaction_id) AS total_transactions,
    ROUND(SUM(fs.revenue)::numeric, 2) AS total_revenue,
    ROUND(
        COUNT(DISTINCT fs.source_transaction_id) * 100.0 /
        SUM(COUNT(DISTINCT fs.source_transaction_id)) OVER ()
        ::numeric, 1) AS pct_of_transactions
FROM fact_sales fs
JOIN dim_payment_method dpm ON fs.dim_payment_method_id = dpm.id
GROUP BY dpm.method
ORDER BY total_transactions DESC;
```

dim_payment_method 1 X

SELECT dpm.method AS payment_method, G | Enter a SQL expression to filter results (use Ctrl+Space)

	A-Z payment_method	123 total_transactions	123 total_revenue	123 pct_of_transactions
1	credit	157,585	7,113,182.35	45
2	debit	105,140	4,735,903.02	30
3	cash	52,452	2,371,089.98	15
4	mobile	34,865	1,578,505.77	10

Also very straightforward, shows the payment methods that our customers use. 85% of the customers use some form of card method whether that be credit, debit, or mobile, that's a sign of the times that we live in, not many people carry cash nowadays. Maybe would it be wise to try and partner with a credit card company to make our own credit card for rewards members to sign up for?

Query 3: revenue by hour of day:

```
SELECT
    dt.hour_of_day,
    dt.time_label,
    dt.part_of_day,
    dt.is_rush_hour,
    COUNT(DISTINCT fs.source_transaction_id) AS total_transactions,
    ROUND(SUM(fs.revenue)::numeric, 2) AS total_revenue,
    ROUND(AVG(fs.revenue)::numeric, 2) AS avg_revenue_per_item
FROM fact_sales fs
JOIN dim_time dt ON fs.dim_time_id = dt.id
GROUP BY dt.hour_of_day, dt.time_label, dt.part_of_day, dt.is_rush_hour
ORDER BY dt.hour_of_day;
```

	hour_of_day	time_label	part_of_day	is_rush_hour	total_transactions	total_revenue	avg_revenue_per_item
1	8	08:00	Morning	[v]	11,889	540,430.07	10.04
2	9	09:00	Morning	[v]	18,264	815,683.22	9.97
3	10	10:00	Morning	[]	14,992	675,549.55	10.01
4	11	11:00	Morning	[]	20,107	905,105.83	10.04
5	12	12:00	Afternoon	[v]	30,302	1,370,341.72	10.05
6	13	13:00	Afternoon	[]	28,536	1,284,317.03	10.03
7	14	14:00	Afternoon	[]	22,201	1,004,695.91	10.04
8	15	15:00	Afternoon	[]	20,256	915,240.04	10.06
9	16	16:00	Afternoon	[]	25,437	1,150,901.3	10.01
10	17	17:00	Evening	[v]	38,757	1,754,255.15	10.03
11	18	18:00	Evening	[v]	43,188	1,942,554.37	10.01
12	19	19:00	Evening	[]	35,761	1,608,876.32	10.02
13	20	20:00	Evening	[]	25,516	1,154,838.52	10.03
14	21	21:00	Evening	[]	14,836	675,892.09	10.06

Shows the revenue by hour of day, this shows our busier hours and our less busy hours. Very straightforward but good information. Hey, let's make sure we are adequately staffed from 5-7 because that's when we see a lot of customers, and we can only have a couple of employees working at 8am and 9pm since not many customers come. Hey, maybe we should bring in that new yogurt company to pass out samples at noon because we see a lot of people come in then.